Ders 14 Çalışma Özeti

Ders 14 Çalışma Özeti

Genel Konular

- Main Memory (Ana Bellek) Yönetimine Giriş
  - İşlemci olmadan programlar çalışamaz, ama programlar bellekte durmalıdır. Bellek, işlemcinin doğrudan eriştiği temel yapı taşıdır.
  - Bu ders: Bellek yönetiminin temelleri, segmentasyon, sayfalama (paging), sanal bellek girişi.
  - İki temel kavram: Spatial locality (mekan yerellik) ve Temporal locality (zamansal yerellik).
- Belleğin Temel Rolü
  - İşlemci veriyi (data) ve instruction'ları register'lardan ve bellekten alır.
  - Bellek, CPU'nun doğrudan erişebildiği iki temel yapıdan biridir (diğeri register'lar).
  - Cache, CPU ile bellek arasında transparan (şeffaf) bir ara katman olarak çalışır; CPU doğrudan load/store yapar, cache hız kazandırır.
- Bellek Controller
  - Bellek için de bir controller vardır (diğer çevre birimleri gibi).
  - İşlemciden sürekli okuma/yazma istekleri gelir; her isteğin adres ve data kısmı vardır.
  - Okuma: adres gönderilir, sonuç data olarak döner.
  - Yazma: adres ve data beraber gönderilir.
- Bellek Hızı Sorunu
  - 2004'te Pentium 3.8 GHz'de çalışıyordu; bugün bile bellek birimleri 2 GHz'e bile yaklaşamıyor.
  - Bu nedenle cache yapısı zorunludur.
- Protection (Koruma) Kavramı
  - Prosesler adres alanı olarak bellekte izoledir.
  - Aynı kullanıcının iki proses'i bile OS'nin belirlediği kurallar olmadan birbirinin belleğine erişemez.
  - Protection, bir process'in kendi bellek alanı dışına erişiminin donanım seviyesinde engellenmesidir.
- Base ve Limit Register
  - İşlemci mimarilerinde bellek koruması için iki özel register tanımlanır.
  - Base register: Process'in bellekteki başlangıç adresi.
  - Limit register: Process'in maksimum erişebileceği bellek boyutu.
  - Her bellek erişiminde: `base ≤ adres < base + limit` kontrolü yapılır. Aksi durumda trap (software interrupt) oluşur.
- Adres Binding (Adres Bağlama)
  - Process'in belleğe yükleneceği yer önceden bilinmez (hangi adreste boş yer varsa oraya).
  - Programcı adres belirleyemez, derleyici de bilemez.
  - Çözüm: Tüm programlar sıfır adresinden başlayacak şekilde derlenir. Relocation (yeniden konumlandırma) ile gerçek yükleme adresine göre offset eklenir.
  - Relocatable: Program sıfır adresine göre derlenir; yüklendiğinde relocation offset eklenir.
- Binding Zamanları
  - **Compile time (Derleme zamanı)**: Mutlak adresler kullanılır (sistem programı, gömülü sistem). Değiştirilemez.
  - **Load time (Yükleme zamanı)**: Loader programı belleğe yüklerken her adrese offset

ekler. Relocatable.
    - **Execution time (Çalışma zamanı)**: Dinamik binding. Shared library, shared object (.so), DLL'ler. Çalışma sırasında adresler bağlanır.
- Mantıksal vs Fiziksel Adres
    - Mantıksal (logical/virtual) adres: Programın ürettiği, CPU'nun gördüğü adres. Sıfırdan başlar.
    - Fiziksel (physical) adres: Bellek pinlerinde görünen gerçek adres.
    - MMU (Memory Management Unit): Mantıksal adresi fiziksel adrese dönüştüren donanım birimi.
    - Relocation register: MMU'da tutulan, mantıksal adrese eklenen değer.
- Statik vs Dinamik Linking
    - **Statik Linking**: Program derlenirken tüm kütüphane fonksiyonları programa eklenir. Program boyutu büyür.
    - **Dinamik Linking**: Kütüphane çalışma sırasında programa bağlanır. Unix'te .so (shared object), Windows'ta .dll.
- Swapping
    - Uzun süre I/O yapacak veya bekleyecek bir process'i bellekten daha iyi yararlanmak için ikinci belleğe (disk) transfer etme.
    - Process control block bellekte kalır; process'in state'i swap out edilmiş duruma getirilir; process'in belleği olduğu gibi diske yazılır.
    - Bekleme bittiğinde swap in: diskten belleğe geri okunur.
    - Roll out/roll in: Öncelik tabanlı scheduling'de düşük öncelikli process çıkarılıp yüksek öncelikli process alınırsa.
    - Backing store: Diskin process'lerin tutulduğu alan.
    - Swapping maliyetli: Process'in tüm belleği dışarı yazılıp tekrar okunmalı.
    - Relocation ile hızlandırılabilir (tek bir relocation register güncellemesi yeterli).
    - Modern OS'lerde normalde kullanılmaz; sadece acil durumlarda (bellek %80-85 dolduğunda) devreye girer.

Hocanın Özellikle Vurguladığı Kısımlar

- Programcının Adres Bilmemesi
    - Hoca vurgular: "Ben bir prosesi çalıştırırken daha doğrusu bir programı çalıştırmak için oluşturacağım prosesi memory de hangi adreste oluşturacağımı önceden biliyor muyum? Hayır bilmiyorum. O zaman program yazarı bu programı hangi adrese göre yazacak? Onun da bilebilmesine imkan yok."
- Sıfır Adres Kuralı
    - Hoca vurgular: "O zaman en akıllıca olan adres bütün programların sıfır adresinden başlaması. Çünkü sıfır adresinden başladığımız zaman programın içerisindeki diğer bütün adresler sıfıra göre relatif adresler olacak. Yani eğer ben bu programı gerçek sistemde sıfır adresinden değil de bin adresinden başlatmak zorunda kalırsam, ki olabilir az önceki örneği gördünüz, o zaman bütün adreslere ne yapmam lazım? Bin değerini eklemem lazım ve bir çırpıda tüm adresler düzelir."
- Static vs Dinamik Linking Farkı
    - Hoca vurgular: "Eğer static linking yaparsanız programınıza matematik kütüphanesi olduğu gibi eklenecek yani programınızın boyutu büyüyecek. Halbuki siz onun içerisinden sadece neyi kullandınız? Cosunius'u kullandınız. Yani static linking sırasında bir kütüphanenin içerisinden sadece programın ihtiyaç duyduğu fonksiyonu metodu alayım onu programa ekleyeyim. Geri kalanını dışarıda bırakayım gibi bir şey söz konusu değil ne yazık ki. O kütüphanenin tamamen programın içerisinde gömülmesi gerekiyor."
- Cheat Engine Sorusuna Cevap
    - Hoca açıklar: "Bir debugger mı? Bir şey mi? Simülatör mü? Nedir bu? Tam olarak bilmiyorum da oradan istediğimiz başka bir programı seçip onun memory'sinde arama yapıp değiştirebiliyoruz. Çalışma esnasında mı yoksa offline'da mı? Çalışma esnasında da offline fark etmiyor. Yani offline'de yaparsın zaten. Loader'ın yaptığı gibi ne olacak ki girersin işte bütün adres referansları her türlü şey belirli şey yapabilirsin."
- Windows Swapping Problemi
    - Hoca, Windows'un %80-85 bellek doluluğunda swapping'i tetiklediğini vurgular: "Sizin 16 GB memory'niz var ancak %80'i 12 GB yani 12 GB doldurduktan sonra son 4

GB kullanmak için aslında ciddi bir yer olmasına rağmen işletim sisteminin yapısından ötürü inanılmaz bir Swapping yapıyorsunuz."

- Relocation Avantajı
  - Hoca vurgular: "Ancak relocation register'ımız varsa ve bir memory management unit kullanıyorsak, yapmamız gereken yeni adresin, yeni başlangıç adresinin bu process'i memory'de yerleştirdiğimiz yeni başlangıç adresinin değerini relocation register'a vermek. Yani tek bir değeri update ederek devam edebiliyoruz."

Kısa Tekrar Notları

- Base + limit register: hardware address protection.
- Mantıksal adres (sanal) vs fiziksel adres.
- MMU: mantıksal → fiziksel dönüşüm.
- Binding zamanları: compile, load, execution.
- Statik linking: tüm kütüphane programa gömülür.
- Dinamik linking: .so (Unix), .dll (Windows).
- Swapping: process'in belleği diske yazılır, sonra geri okunur.
- Relocation: sıfır adresine göre derleme + yükleme zamanında offset ekleme.

Detaylı Açıklamalar

Ders 14, main memory (ana bellek) yönetimine giriş niteliğindedir. Bu ders, dönemin son dersidir ve bellek yönetiminin temellerini ele alır.

İşlemci olmadan programlar çalışamaz; ancak programlar da bellekte durmalıdır. İşlemci, register'lar ve bellek üzerinde çalışır. Cache, CPU ile bellek arasında transparan bir ara katmandır; CPU load/store yaparken cache hız kazandırır. Bellek controller'ı, işlemciden gelen okuma/yazma isteklerini yönetir.

Bellek hızı, işlemci hızının çok altındadır. 2004'te Pentium 3.8 GHz'de çalışıyordu; bugün bile bellek birimleri 2 GHz'e bile yaklaşamıyor. Bu nedenle cache yapısı zorunludur.

Bellek koruması (protection), proseslerin birbirlerinin bellek alanlarına izinsiz erişimini engeller. Base ve limit register'lar bu amaçla kullanılır. Base, process'in bellekteki başlangıç adresini; limit, process'in maksimum erişebileceği bellek boyutunu tutar. Her bellek erişiminde donanım seviyesinde `base ≤ adres < base + limit` kontrolü yapılır. İhlal durumunda trap (software interrupt) oluşur ve OS müdahale eder.

Adres binding, mantıksal adreslerin fiziksel adreslere bağlanması sürecidir. Programcı adres belirleyemez (hangi adreste boş yer olacağını bilemez); derleyici de bilemez. Çözüm: tüm programlar sıfır adresinden başlayacak şekilde derlenir (relocatable). Yükleme sırasında relocation offset eklenir. Üç binding zamanı vardır: compile time (mutlak adresler, gömülü sistemler), load time (loader offset ekler, relocatable), execution time (dinamik, paylaşımlı kütüphaneler).

Mantıksal (logical/virtual) adres, programın ürettiği CPU'nun gördüğü adrestir. Fiziksel (physical) adres, bellek pinlerinde görünen gerçek adrestir. MMU (Memory Management Unit), mantıksal adresi fiziksel adrese dönüştüren donanım birimidir. Relocation register, MMU'da tutulan, mantıksal adrese eklenen değerdir.

Statik linking'de program derlenirken tüm kütüphane fonksiyonları programa eklenir. Dinamik linking'de ise kütüphane çalışma sırasında programa bağlanır; Unix'te .so (shared object), Windows'ta .dll olarak adlandırılır. Dinamik linking program boyutunu küçültür ve kütüphane güncellemelerini kolaylaştırır.

Swapping, bellek yönetiminin temel tekniklerinden biridir. Uzun süre I/O yapacak veya bekleyecek bir process'in belleği diske yazılır; process kontrol blok bellekte kalır. Bekleme bittiğinde process geri yüklenir. Swapping maliyetlidir (tüm bellek dışarı yazılıp okunmalı); relocation ile hızlandırılabilir. Modern OS'lerde normalde kullanılmaz; sadece bellek doluluğu eşik değeri aşıldığında (örn. %80-85) devreye girer.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli

çalışabilirsiniz.